

Import Statements

```
In [1]: import custom_cmap
import os
import sys
from os import chdir, path
from glob import glob

from PIL import Image
from IPython.display import display
import matplotlib.pyplot as plt
import matplotlib as mpl
import pandas as pd
from moria import reduce
from astroquery.ogle import Ogle
from astropy.coordinates import SkyCoord
from astropy import units as u
from pathlib import Path
from astropy.visualization import PercentileInterval
from astropy.coordinates import SkyCoord
from astropy import units as u
from re import A

from scipy.signal import find_peaks
from scipy.optimize import curve_fit

import inspect
import numpy as np
#from new_flystar.flystar import match
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt
import matplotlib.path_effects as pe
from matplotlib.ticker import (MultipleLocator, AutoMinorLocator)
import matplotlib.font_manager
import matplotlib.ticker
from matplotlib.ticker import FormatStrFormatter
import pandas as pd

#import smplotlib

from astropy.io import fits
from astropy.visualization import LogStretch, ImageNormalize
import plotly.express as px
import numpy as np
from astropy.visualization import ImageNormalize, AsinhStretch, SqrtStretch,

#import smplotlib

import pdb
```

```
plt.rcParams.update({'font.size':25})
plt.rc('text', usetex=True)
%config InlineBackend.figure_format = 'retina'
%load_ext autoreload
%autoreload 2
```

```
/Users/tmbhadra/miniforge3/lib/python3.12/site-packages/astroquery/ogle/__in
it__.py:40: UserWarning: Experimental: OGLE has not yet been refactored to h
ave its API match the rest of astroquery.
  warnings.warn("Experimental: OGLE has not yet been refactored to have its
"
```

Understanding the main directory.

The directory structure for your data analysis should be organized as follows.

```
00.DATA
01.XYM
02.CMD
03.LOC_TRANS
04.PSF_EXTRACT
05.COORD_TRANS(OPTIONAL)
06.FIT
07.CALIBRATION
```

This directory structure has been set up in "MORIA/data".

All necessary scripts are included within the corresponding folders under MORIA/data. Therefore, the simplest way to run MORIA on your target is to copy all eight folders from MORIA/data to the location where you intend to perform your analysis. Begin by placing your exposures in data/00.DATA.

Note: `chmod +x program.src` is a useful command to run whenever script execution fails due to permission issues

Notebook Goals

This notebook involves manual steps to calibrate HST magnitudes with OGLE-III magnitudes

```
In [2]: #This is the directory where you are processing your data. This does not poi
directory = os.getcwd()
```

Step 0

We assume you ran the `output_stacks.ipynbm`, `cmd_diagram.ipynb`, `creating_psf.ipynb`, `fitting_psf.ipynb` (in that order) notebooks correctly

Step 1: Enter coordinates of your target

The goal here is to calibrate the HST photometry to the OGLE-III database. To obtain location of your event you will first need to obtain the OGLE-III field number, chip number, and pixel coordinates at the OGLE web page using the "OGLE Field Finder" under the "Sky Coverage". MORIA automates this with the scripts below.

Once MORIA does that, it chooses the appropriate OGLE-III catalog file at <http://www.astrouw.edu.pl/ogle/ogle3/maps/blg/maps/>. It will download the appropriate photometry map for your event from here.

For example, consider OGLE-2012-BLG-0563: the OGLE-III catalog file is `blg226.7.map`, and the OGLE reference image corresponding to this photometry catalog file is `blg226.l.7.fts`, which can be obtained from http://www.astrouw.edu.pl/ogle/ogle3/maps/blg/ref_images/.

The routines in this module compare the HST photometry to the OGLE photometry in order to enable the HST photometry to be calibrated to OGLE. Special efforts are required to avoid blending effects, where several HST stars will be merged together to make an object that is considered to be a single star in the lower angular resolution OGLE images. Also, PSF-fitting magnitudes are dependent on the details of the PSF model.

```
In [ ]: ogle_ra = "17:58:17.29" #Edit with the coordinates of your event
        ogle_dec = " -29:03:12.24" #Edit with the coordinates of your event

        ogle_coord = SkyCoord(ogle_ra, ogle_dec, unit = (u.deg, u.deg))
        ogle_ra_deg = np.float64(ogle_coord.ra)
        ogle_dec_deg = np.float64(ogle_coord.dec)

        ogle_band = "I"
```

Step 2: Download the OGLE map and reference image

```
In [ ]: ogle_field_number, ogle_chip_number = reduce.get_chip_number(ogle_ra_deg, ogle_dec_deg)

        reduce.download_ogle_map_and_reference(
            directory=directory,
            ogle_field_number=ogle_field_number,
```

```
In [ ]: ogle_field_number, ogle_chip_number = reduce.get_chip_number(ogle_ra_deg, ogle_dec_deg)

        reduce.download_ogle_map_and_reference(
            directory=directory,
            ogle_field_number=ogle_field_number,
            ogle_chip_number=ogle_chip_number,
            ogle_band=ogle_band,
            destination_subdir="07.CALIBRATION",
            overwrite=False,
        )
```

When calibrating the module below, we require running the program `psf_star_mags_mcmc.xOg` using the script `run_psf_star_lmags_mcmc.src` and the input file `IN_psf_star_mags_mcmc.l`. We will first design the input for this script.

1. The PSF file name (already entered)
2. The input file name tag (already entered)
3. The output file name tag (already entered)
4. The number of Markov Chain steps (e.g. 100,000)
5. The maximum size of MCMC coordinate steps in pixels and the error bar fudge factor (e.g. 0.02)
6. The maximum distance in x and y from the star center for pixels to be included in the fit, and the χ^2 threshold to define an outlier pixel. (e.g. 16)
7. The sky model to use: 0 for annulus, 1 for fit sky on the same pixels as the PSF peak. The annulus sky model is needed for determining the PSF model, but it is more sensitive to blending than the fit sky model. (e.g. 1.0)
8. A list of star numbers to produce PIX_SHOW files for, ending with a 0 to exit the program. These PIX_SHOW are the same as the ones produced in module 06, and they are useful to tell you how well each star was fit. But, they are large files that are time consuming to produce, and it is unlikely that you'll want them for every star. (e.g. 10)

localhost:8889/lab/tree/Documents/Work/NASA/MB22465_4Dex/04.calibrations.ipynb

```
"maximum_distance_x": 2.5,  
"maximum_distance_y": 2.5,  
"chi2cut": 16,  
"sky_model": 1,  
"star_numbers_pix_show": 10,  
}
```

```
In [ ]: reduce.calibration_input_file_one(directory)
```

Step 4: Recreate calibration input for V magnitude

Repeat this procedure for the V band using script `run_psf_star_Vmags_mcmc.src`, which uses the input file `IN_psf_star_mags_mcmc_V`.

```
In [ ]: reduce.calibration_input_file_two(directory)
```

Step 5: Create new calibration map

When you finish running this step, you should see two new files in 07.CALIBRATION:

1. MATCHUP.F814W_cal.XYM
2. MATCHUP.F814W_cal_only.XYM

```
In [ ]: reduce.calibration_new_matchup(directory)
```

Step 6

This is a manual step. Notice that the 07.CALIBRATION Setup has four `IN.*` files in it already. The first is `IN.cal_star_num_2_MATCHUP` which is used to re-create matchup files used for calibration. Then we have `IN.fit_HST_logle_col_1` and `IN.fit_HST_logle_col_2` used in Step 6 below. We also have `IN.VI_HST_ogle_man_match4_backup`.

As of now, you need to manually edit '`IN.VI_HST_ogle_man_match4`'. This step instructs you on how to edit that file.

An important part of this input file is a list of stars that have been matched in the OGLE reference image and the HST image, `outputq_F814W.fits`. At this point, navigate to the 07.CALIBRATION folder using your preferred terminal and run '`python starlist2reg.py`'. You will be prompted to enter the name of your starlist (e.g. '`blg194.1.map`').

Then, open the *.fits file for your map (e.g. 'blg194.1.fits') in DS9. 'starlist2reg.py' created a '.reg' file that you can load to DS9. These will overlay the positions from the OGLE map on your OGLE images. Choose ones that look best by eye and edit 'IN.VI_HST_ogle_man_match4.'

In 'IN.VI_HST_ogle_man_match4', edit the first row to your matching radius in arcseconds; enter the coordinates (format: hh mm ss, without colons) for your event in the second row; change your map number in the seventh row. Then, you need to enter information about the matching calibrated stars. The first two entries are the HST x and y coordinates (found in the MATCHUP file), then the OGLE x and y coordinates (found in the starlist), followed by the HST I magnitudes, and then the OGLE I magnitudes for those stars.

In the OGLE map, the V magnitude is written before the I magnitude. To make it easier to get OGLE x, y, V and I magnitudes, we recommend opening a file called: 'new_{blg_map_name}.map' (e.g. new_blg194.1.map). 'new_{blg_map_name}.map' is a version of the OGLE map that only reports the OGLE x, y, V and I magnitude.s

Loading the '.reg' file into DS9 makes this process quicker.

However, this is the only part of the pipeline that has been left manual. Once the 'IN.VI_HST_ogle_man_match4' file has been created, proceed with the notebook.

```
In [ ]: reduce.calibration_hst_ogle_match(directory)
```

Step 7

The final calibration program is used to select constraints to put on the parameters in the VI_HST_ogle_Cal_matches4.dat file to select the calibration stars and calibrate the HST photometry. This is done program fit_HST_IV_ogle_col.xOg.

This program calculates both I band and V band offsets between the calibrated OGLE photometry and the HST photometry, as well as 2-color fits to calibrate both the HST I and V bands to the OGLE I and V band.

The results of these calibration attempts have been routed to the run_fit_HST_IV_ogle_col_1.log and run_fit_HST_IV_ogle_col_2.log files.

If this step fails, you should open ``VI_HST_ogle_Cal_matches4.dat" and remove the spacing between the columns with headers Vo-Vhfs and lg_c2Vmx

```
In [ ]: reduce.fit_calibration(directory)
```

You can now open the log_files present in 07.CALIBRATION/log_files to note down the OGLE-III calibrated HST magnitudes. The log file of concern will be labelled

'run_fit_VI_HST_ogle_man_match4.log'.

If you'd like to celebrate run the cell below

```
In [ ]: reduce.notebook_complete("Calibration notebook")
```

If you'd like to go further and get an OGLE-calibrated CMD Diagram

This is different from the CMD diagrams produced in 02.CMD. This is the CMD diagram after calibrating your HST magnitudes using the OGLE Bulge Map.

First, run the cell below to establish some helper functions

```
In [3]: def flux_to_mag_error(flux, flux_error):
        return 2.5 / np.log(10) * flux_error / flux

def cal_mag_error(I_inst_err, V_inst_err, color_term):
    """Propagate instrumental mag errors through m_cal = zp + (1-k)*I_inst +
    return np.sqrt((1 - color_term)**2 * I_inst_err**2 + color_term**2 * V_i

# Helper function
def get_average_mag_offsets(n_calib_stars=5):

    data = []
    with open(Path(directory).resolve()/f"07.CALIBRATION/VI_HST_ogle_Cal
        for line in f:
            # Skip empty or comment lines
            if line.strip() == "" or line.startswith("#"):
                continue
            parts = line.strip().split() # splits by any whitespace
            data.append(parts)

    # Check the max number of columns
    max_len = max(len(row) for row in data)
    #print(f"Max columns in any row: {max_len}")

    # Pad short rows (if needed)
    for row in data:
        if len(row) < max_len:
            row.extend([None] * (max_len - len(row)))

    initial_mags = pd.DataFrame(data)
    initial_mags.columns = ['id', 'x', 'y', 'nmat', 'nbmat',

    #print(initial_mags)
    #print("-----")
    #print("HST V and I offsets:")
```

```

#print("-----")

# Step 1: Read the last 10 non-empty, non-comment lines
with open(Path(directory).resolve()/f"07.CALIBRATION/fit_HST_IV_ogle
    lines = [line.strip() for line in f if line.strip() and not line

last_10_lines = lines[-n_calib_stars:]

# Step 2: Split each line on whitespace
data = [line.split() for line in last_10_lines]

# Step 3: Pad rows if they have inconsistent lengths (optional but r
max_len = max(len(row) for row in data)
for row in data:
    if len(row) < max_len:
        row.extend([None] * (max_len - len(row)))

# Step 4: Create a DataFrame
calib_mags = pd.DataFrame(data)

calib_mags.columns = ['id',      'V_ogle',    'V_oglem',    'V_og_err',

#print(calib_mags)

#Next step is to match between the two arrays, then calculate the ma

# Make sure 'id' is the same type in both DataFrames
initial_mags['id'] = initial_mags['id'].astype(str)
calib_mags['id'] = calib_mags['id'].astype(str)

# Build lookup for I_hst1 from initial_mags
ihst1_lookup = initial_mags.set_index('id')['I_hst1'].to_dict()

# Lists to store values
i_hst_diff = []
i_hst1_vals = []

# Loop through calib_mags
for _, row in calib_mags.iterrows():
    row_id = row['id']
    if row['V_hst'] != None:
        i_hst_calib = float(row['I_hst'])

        if row_id in ihst1_lookup:
            i_hst1 = float(ihst1_lookup[row_id])
            diff = i_hst_calib - i_hst1
        else:
            i_hst1 = None
            diff = None

        i_hst1_vals.append(i_hst1)
        i_hst_diff.append(diff)

# Add to calib_mags DataFrame

```



```

calib_mags['I_hst1'] = i_hst1_vals
calib_mags['I_hst_diff'] = i_hst_diff

# Print desired columns
#print(calib_mags[['id', 'I_hst1', 'I_hst', 'I_hst_diff']])
avg_offset_I = np.mean(calib_mags['I_hst_diff'])
print("avg offset I = ", avg_offset_I)
# Compute RMS difference
I_rms_diff = np.sqrt(np.mean(calib_mags['I_hst_diff']))
print("RMS of I offset = ", I_rms_diff)
#print("-----")

# Build lookup for V_hst1 from initial_mags
vhst1_lookup = initial_mags.set_index('id')['V_hst1'].to_dict()

# Lists to store values
v_hst_diff = []
v_hst1_vals = []

# Loop through calib_mags
for _, row in calib_mags.iterrows():
    row_id = row['id']
    #pdb.set_trace()
    if row['V_hst'] != None:
        v_hst_calib = float(row['V_hst'])

        if row_id in vhst1_lookup:
            v_hst1 = float(vhst1_lookup[row_id])
            diff = v_hst_calib - v_hst1
        else:
            v_hst1 = None
            diff = None
        #pdb.set_trace()
        v_hst1_vals.append(v_hst1)
        v_hst_diff.append(diff)

# Add to calib_mags DataFrameq

calib_mags['V_hst1'] = v_hst1_vals
calib_mags['V_hst_diff'] = v_hst_diff

# Print desired columns
#print(calib_mags[['id', 'V_hst1', 'V_hst', 'V_hst_diff']])
avg_offset_V = np.mean(calib_mags['V_hst_diff'])
print("avg offset V = ", avg_offset_V)
# Compute RMS difference
V_rms_diff = np.sqrt(np.mean(calib_mags['V_hst_diff']))
print("RMS of V offset = ", V_rms_diff)

return avg_offset_I, avg_offset_V

def cmd_errors(err_v, err_i):
    """Errors for CMD: x = V-I, y = I (uncorrelated V and I)."""
    return np.hypot(err_v, err_i), err_i

```

What you have found in this notebook are the I- and V- band zeropoints for calibration + the color terms associated with the magnitudes. If you'd like to plot the "n" number of stars you found in 06.FIT on a CMD diagram around this target, follow these steps.

1. Find the flux1, flux2, flux1_error and flux2_error values for the F814W and F606W filters from 06.FIT for your PSF fitting. For example, if the 2star-fit gave you the best PSF fit, go to 06.FIT/F814W/2star-fit/log_files/run_mcmc_expand_average.log and enter flux1_814, flux2_814, flux1_814_error, flux_814_error, A0_814_error.

```
In [4]: flux1_814 = 1955.6444
flux2_814 = 8861.5891
flux3_814 = 17096.5322 - flux1_814 - flux2_814 #If 3star-fit is your best-fit
flux1_814_error = 566.5083
flux2_814_error = 1247.3672
A0_814_error = 145.3691
flux3_814_error = np.sqrt(flux1_814_error**2 + flux2_814_error**2 + A0_814_e
```

2. Repeat for F606W filter. For example, if the 2star-fit gave you the best PSF fit, go to 06.FIT/F606W/2star-fit/log_files/run_mcmc_expand_average.log and enter flux1_606, flux2_606, flux1_606_error, flux_606_error, A0_606_error.

```
In [5]: flux1_606 = 1241.8198
flux1_606 = 1626.06252765
flux2_606 = 5039.3218
flux2_606 = 7360.90744224
flux3_606 = 14966.061 - flux1_606 - flux2_606
flux1_606_error = 628.68438
flux2_606_error = 770.5878
A0_606_error = 198.3982
flux3_606_error = np.sqrt(flux1_606_error**2 + flux2_606_error**2 + A0_606_e
```

```
In [6]: star1_Imag_inst = -2.5*np.log10(flux1_814)
star2_Imag_inst = -2.5*np.log10(flux2_814)
star3_Imag_inst = -2.5*np.log10(flux3_814)

star1_Imag_inst_error = flux_to_mag_error(flux1_814, flux1_814_error)
star2_Imag_inst_error = flux_to_mag_error(flux2_814, flux2_814_error)
star3_Imag_inst_error = flux_to_mag_error(flux3_814, flux3_814_error)

star1_Vmag_inst = -2.5*np.log10(flux1_606)
star2_Vmag_inst = -2.5*np.log10(flux2_606)
star3_Vmag_inst = -2.5*np.log10(flux3_606)

star1_Vmag_inst_error = flux_to_mag_error(flux1_606, flux1_606_error)
star2_Vmag_inst_error = flux_to_mag_error(flux2_606, flux2_606_error)
star3_Vmag_inst_error = flux_to_mag_error(flux3_606, flux3_606_error)
```

3. Now open fit_HST_IV_ogle_col.log in 07.CALIBRATIONS, scroll to the bottom and enter the I_zeropoint (I_0), V_Zeropoint (V_0), I_color_term(last column in row with

I_0) and V_color_term(last column in row with V_0)

```
In [7]: I_zeropoint = 28.805384
        I_color_term = 0.167238
        V_zeropoint = 30.713301
        V_color_term = 1.559603
```

```
In [8]: # Star 1 calibrated magnitudes
        star1_I_mag = I_zeropoint + star1_Imag_inst + I_color_term*(star1_Vmag_inst
        star1_V_mag = V_zeropoint + star1_Imag_inst + V_color_term*(star1_Vmag_inst
        star1_I_mag_error = cal_mag_error(star1_Imag_inst_error, star1_Vmag_inst_err
        star1_V_mag_error = cal_mag_error(star1_Imag_inst_error, star1_Vmag_inst_err

        # Star 2 calibrated magnitudes
        star2_I_mag = I_zeropoint + star2_Imag_inst + I_color_term*(star2_Vmag_inst
        star2_V_mag = V_zeropoint + star2_Imag_inst + V_color_term*(star2_Vmag_inst
        star2_I_mag_error = cal_mag_error(star2_Imag_inst_error, star2_Vmag_inst_err
        star2_V_mag_error = cal_mag_error(star2_Imag_inst_error, star2_Vmag_inst_err

        # Star 3 calibrated magnitudes. Uncomment if required
        star3_I_mag = I_zeropoint + star3_Imag_inst + I_color_term*(star3_Vmag_inst
        star3_V_mag = V_zeropoint + star3_Imag_inst + V_color_term*(star3_Vmag_inst
        star3_I_mag_error = cal_mag_error(star3_Imag_inst_error, star3_Vmag_inst_err
        star3_V_mag_error = cal_mag_error(star3_Imag_inst_error, star3_Vmag_inst_err
```

```
In [9]: print(f'Star 1 calibrated I magnitudes is: I = {star1_I_mag:.4f} +/- {star1_
        print(f'Star 1 calibrated V magnitudes is: V = {star1_V_mag:.4f} +/- {star1_
```

Star 1 calibrated I magnitudes is: I = 20.6107 +/- 0.2712
 Star 1 calibrated V magnitudes is: V = 22.7976 +/- 0.6779

```
In [10]: print(f'Star 2 calibrated I magnitudes is: I = {star2_I_mag:.4f} +/- {star2_
        print(f'Star 2 calibrated V magnitudes is: V = {star2_V_mag:.4f} +/- {star2_
```

Star 2 calibrated I magnitudes is: I = 18.9703 +/- 0.1287
 Star 2 calibrated V magnitudes is: V = 21.1587 +/- 0.1968

```
In [11]: print(f'Star 3 calibrated I magnitudes is: I = {star3_I_mag:.4f} +/- {star3_
        print(f'Star 3 calibrated V magnitudes is: V = {star3_V_mag:.4f} +/- {star3_
```

Star 3 calibrated I magnitudes is: I = 19.3195 +/- 0.2007
 Star 3 calibrated V magnitudes is: V = 21.3015 +/- 0.3166

If the values above look suspicious to you (like 2 stars have very close fluxes and magnitudes), we recommend using the best-fit flux values over the MCMC-averaged flux value. You can find the best-fit flux values in

uvp2tri_scon_fsky_I_KeckNOcon.05.final_fit in 06.FIT/3star-fit (Replace the 3star-fit with whatever star-fit you are using). Inside uvp2tri_scon_fsky_I_KeckNOcon.05.final_fit, you will find F1MIN and F2MIN.

Flux_1 = F1MIN * A0 (A0 is found in log_files/run_mcmc_expand_average.log)

Flux_2 = F2MIN * A0

For a 3star-fit, $\text{Flux}_3 = A0 - \text{Flux}_1 - \text{Flux}_2$

```
In [12]: # Instrumental magnitude errors (from flux uncertainties)
print('Instrumental magnitude errors:')
print(f' Star 1: I = +/- {star1_Imag_inst_error:.4f}, V = +/- {star1_Vmag_inst_error:.4f}')
print(f' Star 2: I = +/- {star2_Imag_inst_error:.4f}, V = +/- {star2_Vmag_inst_error:.4f}')
print(f' Star 3: I = +/- {star3_Imag_inst_error:.4f}, V = +/- {star3_Vmag_inst_error:.4f}')
```

Instrumental magnitude errors:

```
Star 1: I = +/- 0.3145, V = +/- 0.4198
Star 2: I = +/- 0.1528, V = +/- 0.1137
Star 3: I = +/- 0.2382, V = +/- 0.1842
```

Enter the ogle map in the cell below. For instance, in 07,CALIBRATION, if you have downloaded BLG205.1.map, you will enter the string 'BLG205.1.map' below:

```
In [13]: ogle_cmd_map = 'BLG205.1.map'
```

Enter the I and V zero-points from run_mcmc_expand_average.log. And enter the name of your target.

```
In [15]: I_zp = I_zeropoint #I-band zeropoint (F814W)
V_zp = V_zeropoint #V-band zeropoint (F606W)
target = 'KMT-2022-BLG-1923' # Edit with your target name
```

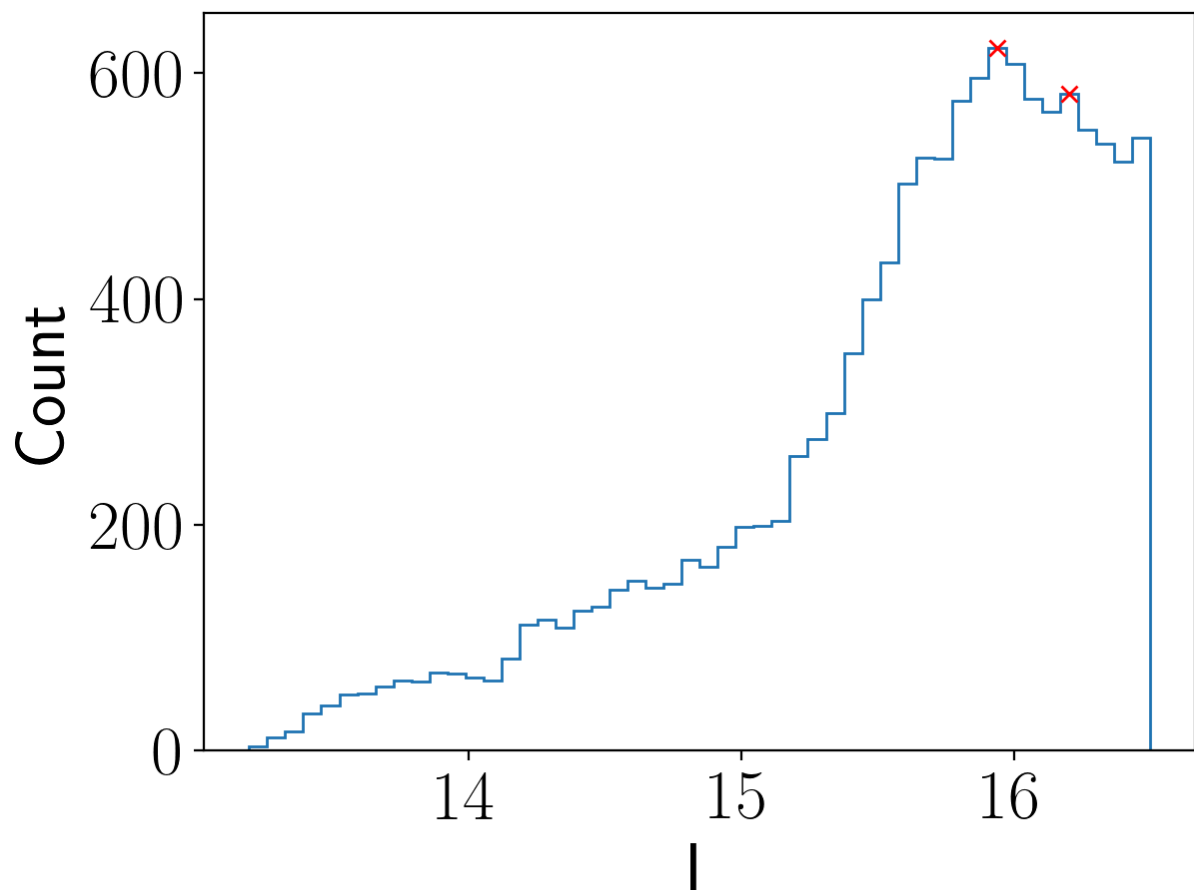
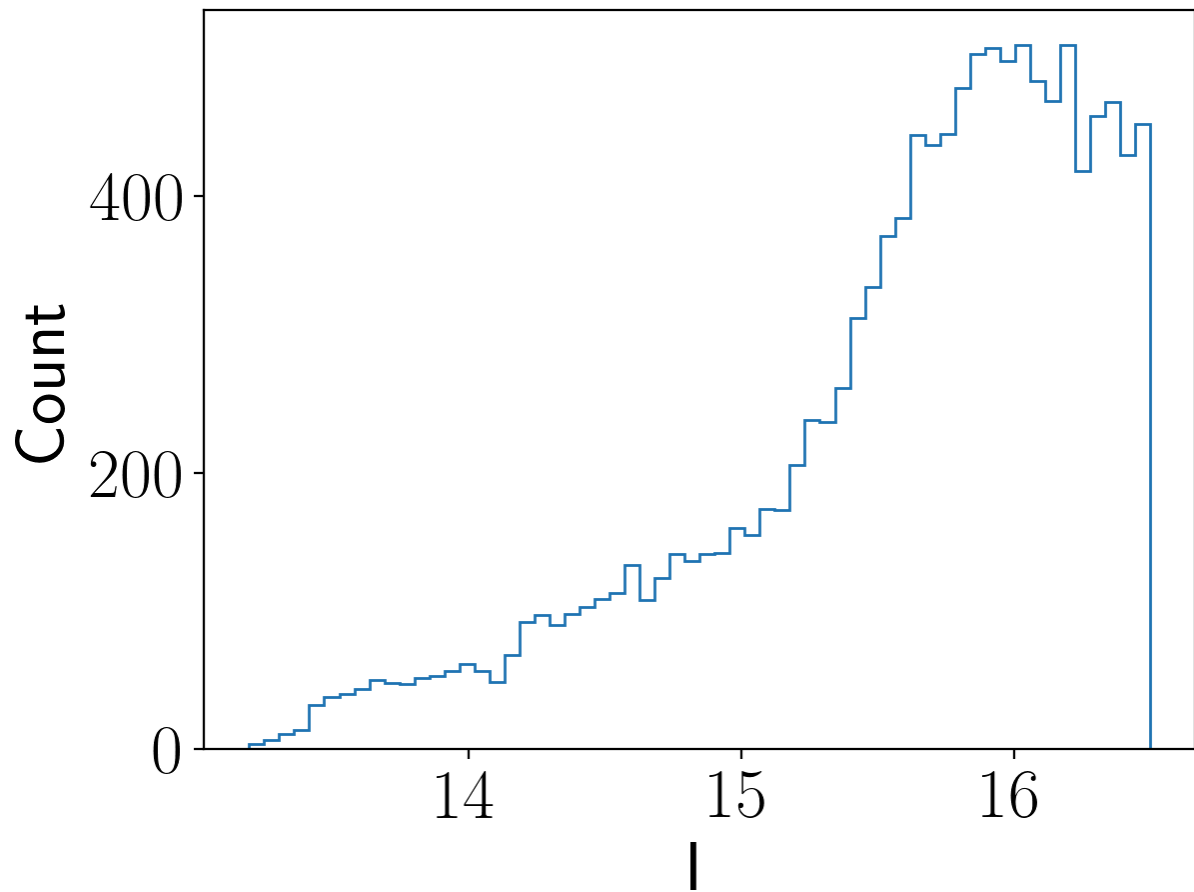
We will also now find the Red Clump I magnitude (RC_Imag) and Red Clump V-I (RC_VI).

We will create a histogram of the OGLE I magnitude. By studying the peaks of the histogram, you can determine, the RC_Imag.

```
In [16]: map_path = Path(directory).resolve()/f"07.CALIBRATION/{ogle_cmd_map}"
ogle_mag = pd.read_csv(map_path, header=None, sep=r"\s+", usecols=[5, 7], na_values=[None])
ogle_mag = ogle_mag[ogle_mag["I"] <= 16.5]
ogle_mag = ogle_mag[ogle_mag["V"] <= 22]
counts, bins = np.histogram(ogle_mag["I"], bins=60) #Alter for more precision
plt.stairs(counts, bins)
plt.xlabel("I")
plt.ylabel("Count")

# Histogram
counts, bins = np.histogram(ogle_mag["I"], bins=50)
centers = 0.5 * (bins[:-1] + bins[1:])
peaks, _ = find_peaks(counts, prominence=10) #Alter prominence more precisely
plt.figure()
plt.stairs(counts, bins)
plt.plot(centers[peaks], counts[peaks], "rx")
plt.xlabel("I")
plt.ylabel("Count")
```

```
Out[16]: Text(0, 0.5, 'Count')
```



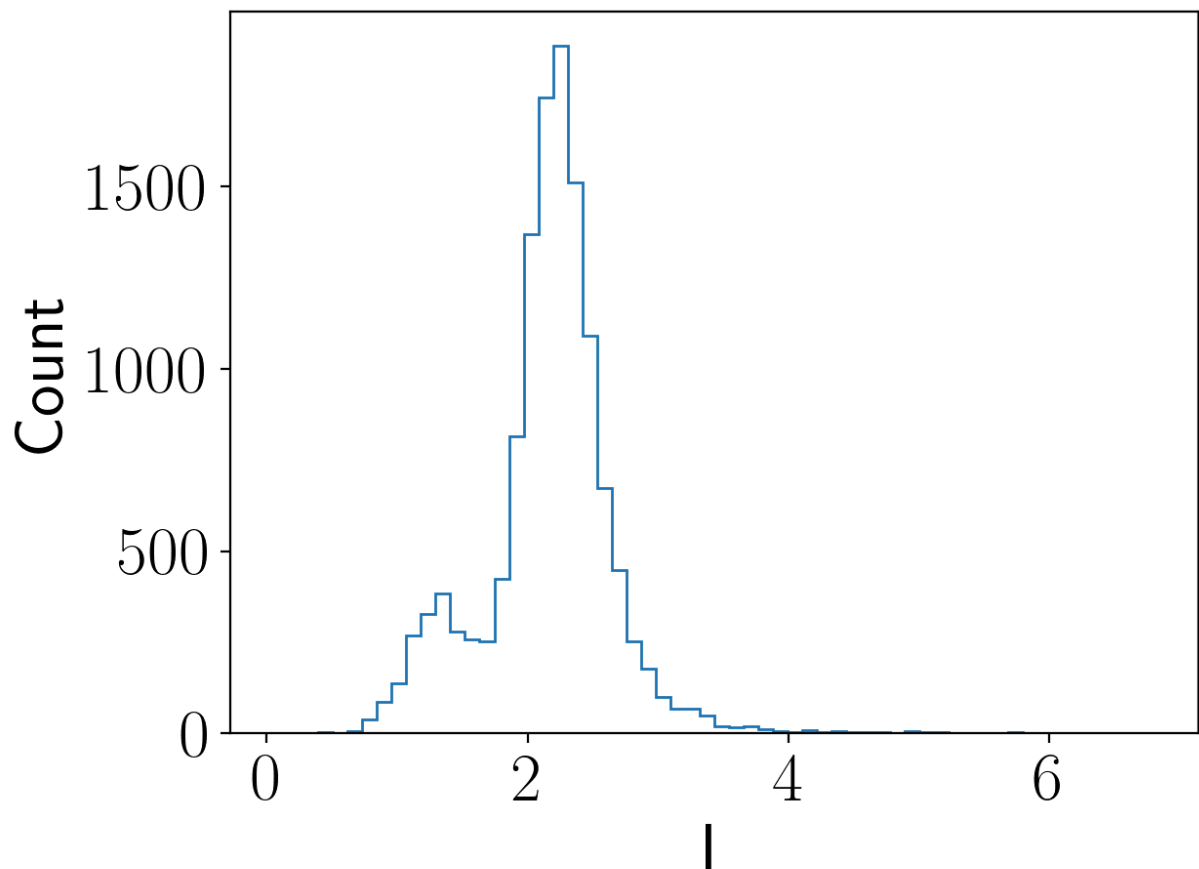
We will create a histogram of the OGLE V-I color. By studying the peaks of the histogram, you can determine, the RC_VI.

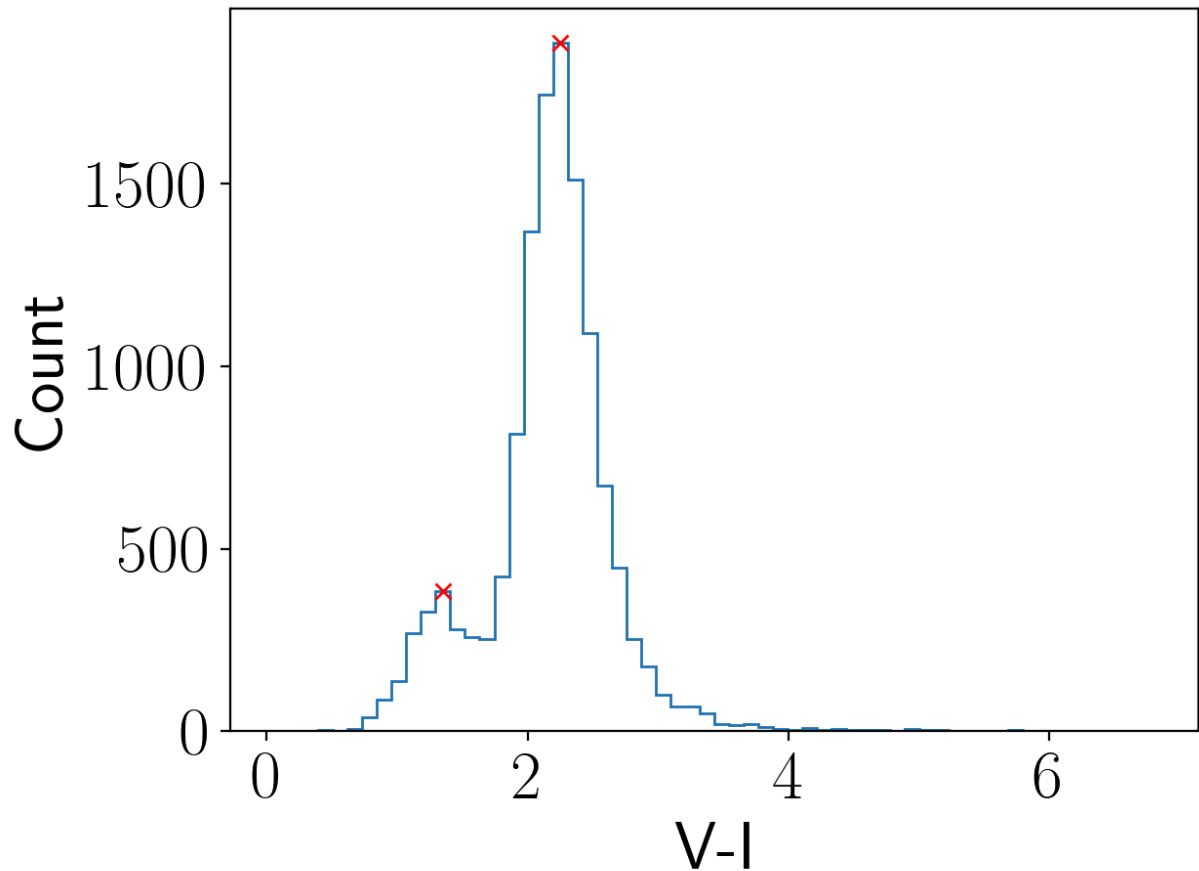
```
In [17]: counts, bins = np.histogram(ogle_mag["V"] - ogle_mag["I"], bins=60)
plt.stairs(counts, bins)
plt.xlabel("I")
plt.ylabel("Count")

# Histogram
counts, bins = np.histogram(ogle_mag["V"]-ogle_mag["I"], bins=60)
centers = 0.5 * (bins[:-1] + bins[1:])
peaks, _ = find_peaks(counts, prominence=50)

plt.figure()
plt.stairs(counts, bins)
plt.plot(centers[peaks], counts[peaks], "rx")
plt.xlabel("V-I")
plt.ylabel("Count")
```

```
Out[17]: Text(0, 0.5, 'Count')
```





```
In [18]: RC_Imag = 15.9 # Can get this from Nataf+2013 or OGLE Extinction Calculator
RC_VI = 2.25 #Also from Nataf+2013 or OGLE Ext. Calc.
```

```
In [19]: #####
#OPEN HST and OGLE FILES#
#####

hst_I = np.genfromtxt(Path(directory).resolve()/f"07.CALIBRATION/MATCHUP.F81
hst_V = np.genfromtxt(Path(directory).resolve()/f"07.CALIBRATION/MATCHUP.F60
ogle_cmd = np.genfromtxt(Path(directory).resolve()/f"07.CALIBRATION/{ogle_cr
I_offset, V_offset = get_average_mag_offsets()

#####
#Load Star 1#
#####
star1_I = star1_I_mag
err_neighbor_I = star1_I_mag_error

star1_V = star1_V_mag
err_star1_V = star1_V_mag_error

offset_star1_vi = star1_Vmag_inst_error - star1_Imag_inst_error
offset_star1_i = star1_Imag_inst_error
offset_star1_v = star1_Vmag_inst_error

#####
```

```

#Load Star 2#
#####
star2_I = star2_I_mag
err_star2_I = star2_I_mag_error

star2_V = star2_V_mag
err_star2_V = star2_V_mag_error

offset_star2_vi = star2_Vmag_inst_error - star2_Imag_inst_error
offset_star2_i = star2_Imag_inst_error
offset_star2_v = star2_Vmag_inst_error

#####
#Load Star 3 (Comment out if there is no star 3)#
#####
star3_I = star3_I_mag
err_star3_I = star3_I_mag_error

star3_V = star3_V_mag
err_star3_V = star3_V_mag_error

offset_star3_vi = star3_Vmag_inst_error - star3_Imag_inst_error
offset_star3_i = star3_Imag_inst_error
offset_star3_v = star3_Vmag_inst_error

#####
#Color Palette#
#####

COLORS = {
    'field': 'k',          # lighter gray – lets foreground pop
    'calibrated': '#00e8a4', # vivid mint/aqua – HST calib stars
    'red_clump': '#ff2d55',  # hot pink-red
    'star2': '#ff9f0a',      # bright orange
    'star3': '#0a84ff',      # vivid blue
    'star1': '#bf5af2',      # bright violet
    'edge': '#1a1a1a',
}

MARKER_KW = dict(s=85, marker='o', linewidths=1.0, zorder=100)
EBAR_KW = dict(lw=2.0, capsize=5, capthick=1.5, zorder=99)

hst_vi = ((hst_V[:, 2] + V_zp + V_offset) - (hst_I[:, 2] + I_zp + I_offset))
hst_i = hst_I[:, 2] + I_zp + I_offset
star2_xerr, star2_yerr = cmd_errors(err_star2_V, err_star2_I)
star3_xerr, star3_yerr = cmd_errors(err_star3_V, err_star3_I)
star1_xerr, star1_yerr = cmd_errors(err_star1_V, err_neighbor_I)
star2_vi, star2_i, star2_v = star2_V - star2_I + offset_star2_vi, star2_I +
star3_vi, star3_i, star3_v = star3_V - star3_I + offset_star3_vi, star3_I +
star1_vi, star1_i, star1_v = star1_V - star1_I + offset_star1_vi, star1_I +

```



```

#####
#Plot#
#####

fig, ax = plt.subplots(figsize=(6, 6))

ax.scatter(ogle_cmd[:, 1], ogle_cmd[:, 2], s=0.4, color=COLORS['field'],
            marker='.', alpha=0.8, zorder=1, rasterized=True)

ax.scatter(hst_vi, hst_i, s=28, color=COLORS['calibrated'], marker='.',
            alpha=0.95, zorder=5, edgecolors='none')

ax.scatter(RC_VI, RC_Imag, color=COLORS['red_clump'], edgecolors=COLORS['edge'],
            label='Red Clump', **MARKER_KW)

ax.scatter(star1_vi, star1_i, color=COLORS['star1'], edgecolors=COLORS['edge'],
            label='Star 3', **MARKER_KW)
ax.errorbar(star1_vi, star1_i, xerr=star1_xerr, yerr=star1_yerr,
            color=COLORS['star1'], **EBAR_KW)

ax.scatter(star2_vi, star2_i, color=COLORS['star2'], edgecolors=COLORS['edge'],
            label='Star 2', **MARKER_KW)
ax.errorbar(star2_vi, star2_i, xerr=star2_xerr, yerr=star2_yerr,
            color=COLORS['star2'], **EBAR_KW)

ax.scatter(star3_vi, star3_i, color=COLORS['star3'], edgecolors=COLORS['edge'],
            label='Star 1', **MARKER_KW)
ax.errorbar(star3_vi, star3_i, xerr=star3_xerr, yerr=star3_yerr,
            color=COLORS['star3'], **EBAR_KW)

ax.xaxis.set_ticks_position('both')
ax.yaxis.set_ticks_position('both')
ax.set_xlim(0.1, 4.4)
ax.set_ylim(14, 23.1)

ax.set_ylabel('I', fontsize=18)
ax.set_xlabel('V - I', fontsize=18)
ax.set_title(target, fontsize=18, pad=10)

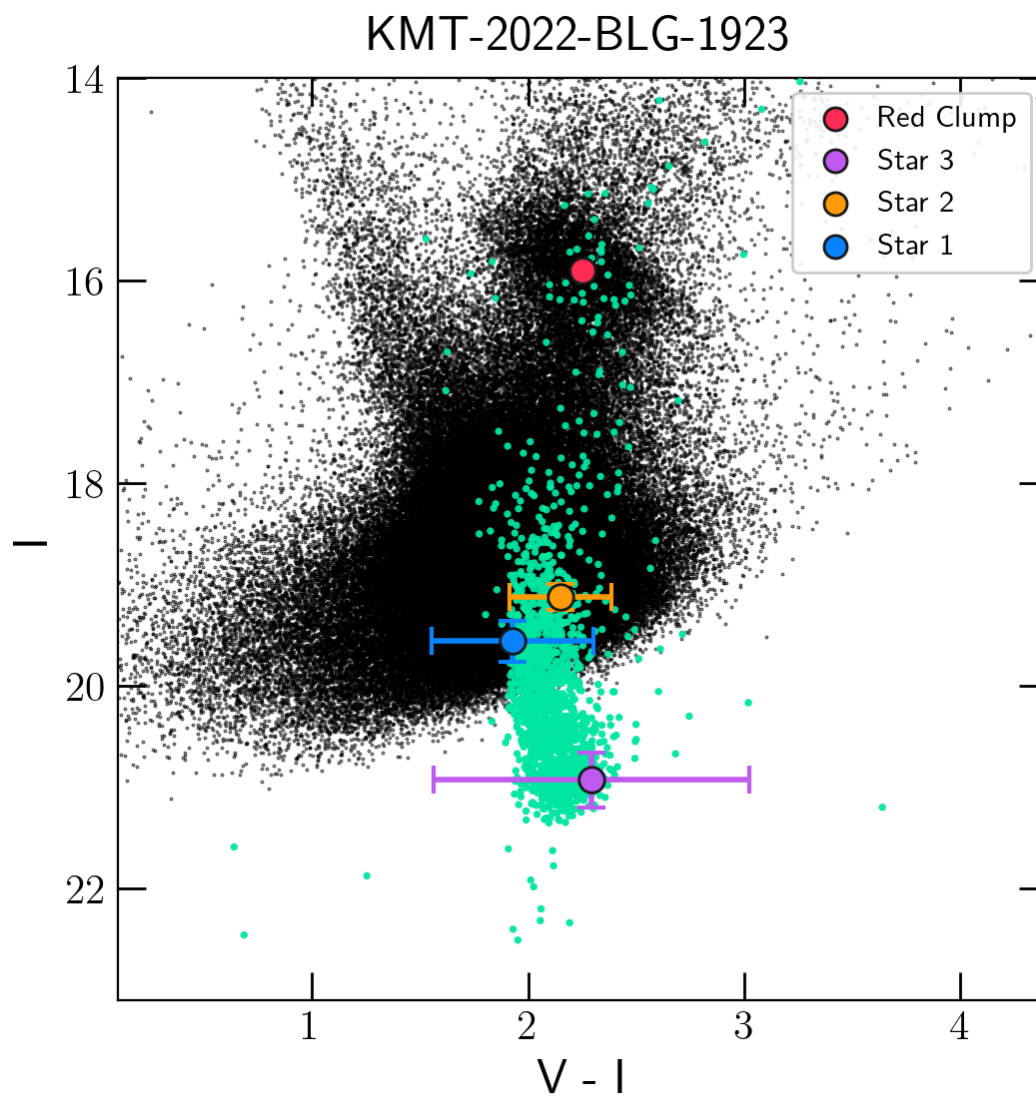
ax.tick_params(which='major', length=10, width=1, labels=15,
               direction='in', right=True, top=True)
ax.tick_params(which='minor', length=5, width=1,
               direction='in', right=True, top=True)
ax.invert_yaxis()

ax.legend(markerscale=0.9, loc='upper right', handletextpad=0.4,
          framealpha=0.92, edgecolor='#cccccc', fontsize=11)

plt.savefig(str(target)+'_CMD.png', dpi=500)
plt.savefig(str(target)+'_CMD.pdf')
#plt.show()

```

```
avg offset I = 0.1581999999999973  
RMS of I offset = 0.3977436360270265  
avg offset V = 0.3556000000000001  
RMS of V offset = 0.596322060635023
```



In []: